Университет ИТМО

Факультет Программной Инженерии и Компьютерной техники

Системы искусственного интеллекта

Лабораторная работа №2

Regularization и KNN

Выполнил:
Ткачев Денис Владимирович
Группа: P3311
ИСУ: 467727
Преподаватель:
Королёва Ю. А.

Санкт-Петербург
2026г.

Содержание

1. Цель работы 3
2. Задание 3
3. Общий подход 3
4. Часть A. Regularization 4
   4.1. Штрафы и градиент 4
   4.2. Условия эксперимента 6
   4.3. Подбор λ 6
   4.4. Итоговые метрики 8
   4.5. Веса и почти нулевые коэффициенты 8
   4.6. Вывод по части A 10
5. Часть B. KNN Classification 11
   5.1. Данные 11
   5.2. Preprocessing 12
   5.3. Класс KNN 12
   5.4. Подбор k 13
   5.5. Confusion matrix 14
   5.6. Вывод по части B 15
6. Часть C. KNN Regression 16
   6.1. Данные 16
   6.2. Preprocessing 16
   6.3. Подбор k 17
   6.4. Результаты 17
   6.5. Вывод по части C 18
7. Вывод 18

1. Цель работы

Добавить L1, L2 и ElasticNet регуляризацию в модели из лабораторной работы №1 и оценить её влияние. Реализовать метод k ближайших соседей для классификации и регрессии и исследовать влияние `k`, метрики расстояния и масштабирования признаков.

2. Задание

	Часть A	**Часть B**	**Часть C**
Модель	Linear и Logistic Regression из лабораторной №1	KNN, классификация	KNN, регрессия
Датасет	Bike Sharing Dataset и Spaceship Titanic	Dry Bean Dataset	Seoul Bike Sharing Demand
Целевая	`cnt` и `Transported`	`Class` - сорт фасоли, 7 классов	`Rented Bike Count` - аренд за час
Что сравнить	без регуляризации, L1, L2, ElasticNet	`k`, Euclidean и Manhattan, со scaling и без	`k`, Euclidean и Manhattan, со scaling и без, baseline
Метрики	MAE, RMSE, R^2 и Accuracy, Precision, Recall, F1, LogLoss	Accuracy, Macro F1, confusion matrix	MAE, RMSE, R^2

Ограничения: все алгоритмы реализуются самостоятельно, готовые Ridge, Lasso, ElasticNet, KNeighborsClassifier, KNeighborsRegressor и другие модели sklearn запрещены. Из sklearn разрешено использовать только `sklearn.metrics`.

3. Общий подход

Код написан на PyTorch, как в лабораторной №1. `torch.nn` и `torch.optim` не используются.

Данные каждой части делятся своей функцией на train `60%`, validation `20%` и test `20%`. Индексы перемешиваются `torch.randperm` с фиксированным `seed=23`. По validation подбираются λ и `k`, test используется один раз для итоговой оценки. Если подбирать параметры по test, его метрика перестаёт быть оценкой на новых данных.

Стандартизация считает среднее и стандартное отклонение только по train и применяет их ко всем трём выборкам, чтобы информация из validation и test не попала в обучение.

```
def train_val_test_split(X, y, val_size=0.2, test_size=0.2, seed=RANDOM_STATE):
    generator = torch.Generator().manual_seed(seed)
    idx = torch.randperm(X.shape[0], generator=generator) # случайная перестановка индексов
объектов [0, n-1]
    n_test = int(X.shape[0] * test_size)
    n_val = int(X.shape[0] * val_size)
```

```
    test_idx, val_idx, train_idx = idx[:n_test], idx[n_test:n_test + n_val], idx[n_test +
n_val:]
    return X[train_idx], X[val_idx], X[test_idx], y[train_idx], y[val_idx], y[test_idx]


def standardize(X_train, X_val, X_test, cols):
    mean = X_train[:, cols].mean(dim=0)
    std = X_train[:, cols].std(dim=0)
    scaled = []
    for X in (X_train, X_val, X_test):
        X_s = X.clone()
        X_s[:, cols] = (X[:, cols] - mean) / std
        scaled.append(X_s)
    return scaled


def evaluate_reg(y_true, y_pred):
    return {
        'MAE': mean_absolute_error(y_true, y_pred),
        'RMSE': mean_squared_error(y_true, y_pred) ** 0.5,
        'R2': r2_score(y_true, y_pred),
    }
```

4. Часть A. Regularization

4.1. Штрафы и градиент

К функции потерь L из лабораторной №1 добавляется штраф на веса `w`: для линейной регрессии L - это MSE, для логистической - BCE. Свободный член `b` не штрафуется.

Вариант	**Loss**	**Добавка к градиенту по `w`**
без регуляризации	L	0
L1	$L + \lambda \sum_j \lvert w_j \rvert$	$\lambda \cdot \operatorname{sign}(w)$
L2	$L + \lambda \sum_j w_j^2$	$2\lambda w$
ElasticNet	$L + \lambda\left(\alpha \sum_j \lvert w_j \rvert + (1-\alpha) \sum_j w_j^2\right)$	$\lambda(\alpha \cdot \operatorname{sign}(w) + 2(1-\alpha)w)$

Здесь λ - сила штрафа `lam`, α - доля L1 в ElasticNet `l1_ratio`. Градиент L считает `autograd`, градиент штрафа добавляется явно в функции `regularized_step`:

1. Шаг по градиенту L и L2-части: $z = w - \eta(\nabla L + 2\lambda_2 w)$, где η - скорость обучения `lr`.
2. Шаг L1-части: вес сдвигается к нулю на $\eta\lambda_1$, но не переходит через ноль: $w = \operatorname{sign}(z) \cdot \max(\lvert z \rvert - \eta\lambda_1, 0)$.

У $\lvert w \rvert$ нет производной в нуле. При обычном шаге $-\eta\lambda_1 \operatorname{sign}(w)$ вес, который должен стать нулём, каждую эпоху перескакивает через ноль и колеблется с амплитудой $\eta\lambda_1$. Обрезка в нуле, она же soft-thresholding, оставляет такой вес ровно нулём. Для веса, который через ноль не переходит, шаг совпадает с шагом по градиенту из таблицы.

```
REGS = ['none', 'l1', 'l2', 'elasticnet']


def penalty_coefs(reg, lam, l1_ratio):
    if reg == 'l1':
        return lam, 0.0
    if reg == 'l2':
        return 0.0, lam
    if reg == 'elasticnet':
        return lam * l1_ratio, lam * (1 - l1_ratio)
    return 0.0, 0.0


def penalty(w, reg, lam, l1_ratio):
    l1, l2 = penalty_coefs(reg, lam, l1_ratio)
    return l1 * w.abs().sum() + l2 * (w ** 2).sum()


def regularized_step(w, grad, lr, reg, lam, l1_ratio):
    l1, l2 = penalty_coefs(reg, lam, l1_ratio)
    z = w - lr * (grad + 2 * l2 * w) # шаг по градиенту loss + градиенту L2
    return torch.sign(z) * torch.clamp(z.abs() - lr * l1, min=0) # шаг L1 к нулю на lr * l1 без
перехода через ноль
```

В `fit` обеих моделей вместо `w -= lr * w.grad` из лабораторной №1 вызывается `regularized_step`. В историю loss на train записывается loss вместе со штрафом, на validation - без штрафа, чтобы варианты можно было сравнивать.

```
class LinearRegression:
    def __init__(self, lr=0.1, n_epochs=2000, reg='none', lam=0.0, l1_ratio=0.5):
        self.lr = lr
        self.n_epochs = n_epochs
        self.reg = reg
        self.lam = lam
        self.l1_ratio = l1_ratio
        self.w = None
        self.b = None
        self.loss_history = []
        self.val_loss_history = []

    def fit(self, X, y, X_val=None, y_val=None):
        self.w = torch.zeros(X.shape[1], dtype=torch.float32, requires_grad=True)
        self.b = torch.zeros(1, dtype=torch.float32, requires_grad=True)
        self.loss_history = []
        self.val_loss_history = []

        for _ in range(self.n_epochs):
            y_pred = X @ self.w + self.b
            loss = ((y_pred - y) ** 2).mean()
            loss.backward() # градиент MSE через autograd

            with torch.no_grad():
                self.loss_history.append(loss.item() + penalty(self.w, self.reg, self.lam,
self.l1_ratio).item())
                self.w.copy_(regularized_step(self.w, self.w.grad, self.lr, self.reg, self.lam,
self.l1_ratio))
                self.b -= self.lr * self.b.grad # b не штрафуется

            self.w.grad.zero_()
```

```
            self.b.grad.zero_()

            if X_val is not None:
                with torch.no_grad():
                    self.val_loss_history.append(((X_val @ self.w + self.b - y_val) **
2).mean().item())

        return self

    def predict(self, X):
        with torch.no_grad():
            return X @ self.w + self.b
```

4.2. Условия эксперимента

Данные и preprocessing взяты из лабораторной №1: для Bike Sharing `29` признаков, для Spaceship Titanic `26`. Отличие одно: целевая переменная `cnt` тоже стандартизирована по train. Без этого MSE измеряется в миллионах, а веса в сотнях, и λ для двух моделей пришлось бы подбирать на разных шкалах. Метрики считаются после обратного преобразования прогноза в число поездок.

Параметр	**Значение**
объекты train / validation / test	Bike Sharing: `439` / `146` / `146`, Spaceship Titanic: `5217` / `1738` / `1738`
обучение	`lr=0.1` для линейной, `lr=0.5` для логистической регрессии, `2000` эпох, нулевые начальные веса
`lam`	`1e-4`, `3e-4`, `1e-3`, `3e-3`, `1e-2`, `3e-2`, `0.1`, `0.3`, `1`
`l1_ratio` для ElasticNet	`0.2`, `0.5`, `0.8`
критерий выбора λ	RMSE на validation для линейной, LogLoss на validation для логистической регрессии

Все четыре варианта обучаются на одном split и одном preprocessing с одинаковыми `lr`, числом эпох и инициализацией.

4.3. Подбор λ

На левых графиках метрика на validation в зависимости от λ, пунктир - модель без регуляризации. На правых - loss на validation по эпохам для лучшего λ каждого варианта.

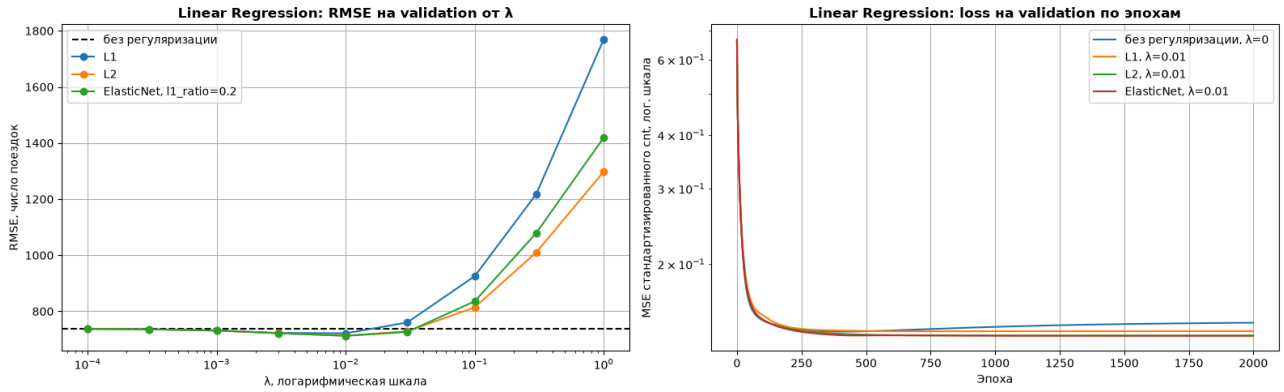


Рис. 1. Linear Regression: RMSE на validation от λ и loss по эпохам

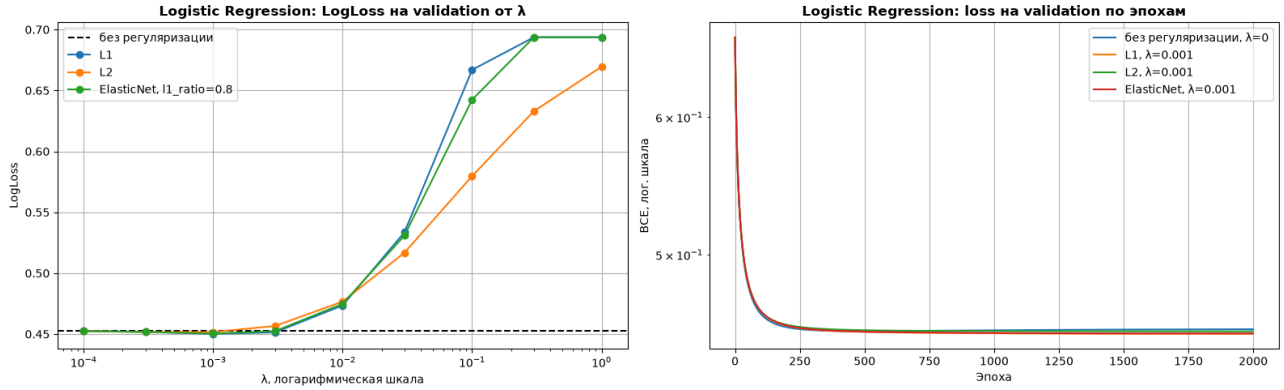


Рис. 2. Logistic Regression: LogLoss на validation от λ и loss по эпохам

У линейной регрессии без регуляризации MSE на validation примерно после `500`-й эпохи начинает расти, это переобучение. С регуляризацией кривые остаются на минимуме. У логистической регрессии кривые почти совпадают.

До λ=`0.01` у линейной и до λ=`0.001` у логистической регрессии метрика на validation снижается, дальше растёт: модель недообучается. С λ=`0.03` метрика L2 растёт медленнее, чем у L1: L1 обнуляет и полезные признаки, а L2 только уменьшает их веса.

Вариант	**λ, линейная**	**RMSE val**	**λ, логистическая**	**LogLoss val**
без регуляризации	`0`	`738.2`	`0`	`0.4529`
L1	`0.01`	`721.7`	`0.001`	`0.4502`
L2	`0.01`	`713.2`	`0.001`	`0.4517`
ElasticNet	`0.01`, `l1_ratio=0.2`	`712.6`	`0.001`, `l1_ratio=0.8`	`0.4504`

4.4. Итоговые метрики

Linear Regression	**MAE train**	**RMSE train**	**R² train**	**MAE test**	**RMSE test**	**R² test**
без регуляризации	549.3	737.5	0.8541	580.1	870.1	0.8063
L1	573.1	763.7	0.8436	590.6	854.0	0.8134
L2	568.1	757.5	0.8461	582.6	853.8	0.8135
ElasticNet	568.2	758.5	0.8457	584.3	854.0	0.8134

Logistic Regression, test	**Accuracy**	**Precision**	**Recall**	**F1**	**LogLoss**	**Accuracy train**
без регуляризации	0.7808	0.7658	0.8023	0.7836	0.4550	0.7962
L1	0.7860	0.7717	0.8058	0.7884	0.4515	0.7968
L2	0.7860	0.7760	0.7977	0.7867	0.4512	0.7959
ElasticNet	0.7860	0.7735	0.8023	0.7877	0.4511	0.7955

Все три варианта регуляризации улучшили test одинаково: RMSE на 16, примерно 2%, LogLoss на 0.004. У линейной регрессии RMSE на train при этом вырос с 737.5 до 757.5: модель меньше подстраивается под train и лучше работает на новых данных.

4.5. Веса и почти нулевые коэффициенты

Коэффициент считается почти нулевым, если $|w_j| < 0.01$. У линейной регрессии веса измеряются в стандартных отклонениях cnt, у логистической - в логитах, поэтому порог для обеих моделей одного порядка.

Вариант	**Модель**	$\sum\|w\|$	$\max\|w\|$	$\|w\| < 0.01$	$w = 0$	**Признаки с $w = 0$**
без регуляризации	линейная	8.09	1.18	0	0	-
L1	линейная	5.48	1.02	12	12	holiday, mnth_2, mnth_3, mnth_4, mnth_6, mnth_8, mnth_11, mnth_12, weekday_2… weekday_5
L2	линейная	6.27	1.00	1	0	-
ElasticNet	линейная	6.00	1.00	1	1	weekday_5
без регуляризации	логистическая	17.85	2.38	0	0	-
L1	логистическая	14.43	2.18	7	7	Destination_Unknown, VIP_True, VIP_Unknown, Deck_F, Deck_T, Deck_Unknown, Side_Unknown
L2	логистическая	13.96	1.83	1	0	-
ElasticNet	логистическая	14.17	2.09	7	7	те же, что у L1

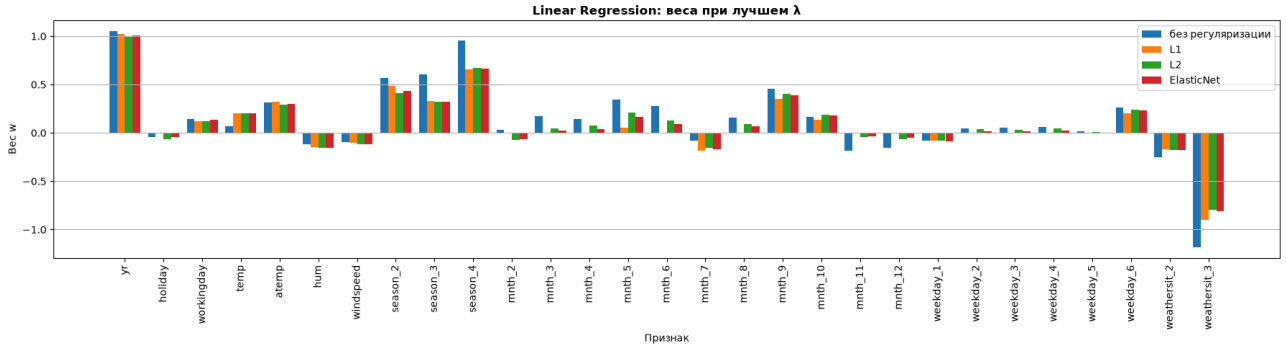


Рис. 3. Linear Regression: веса при лучшем λ

Влияние каждого штрафа видно по всей сетке λ: число весов с $|w| < 0.01$ и сумма $|w|$.

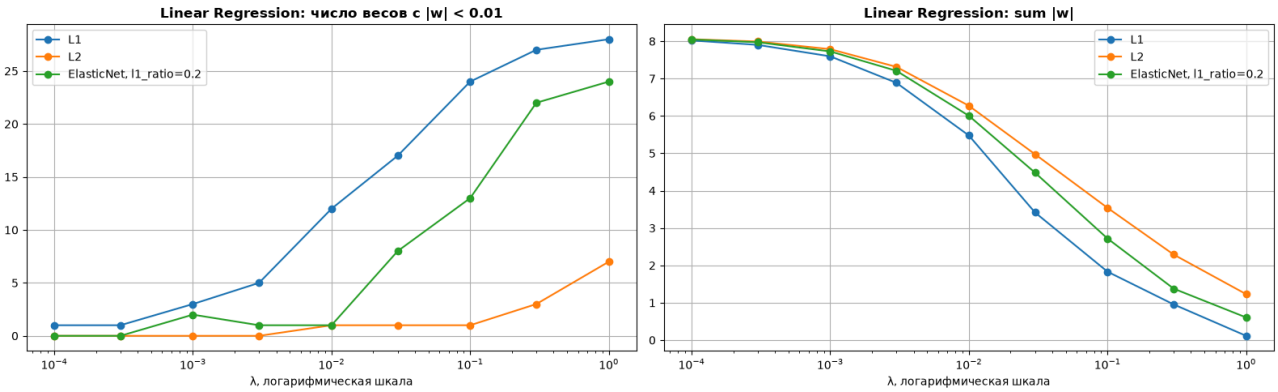


Рис. 4. Linear Regression: число почти нулевых весов и $\sum|w|$ от λ

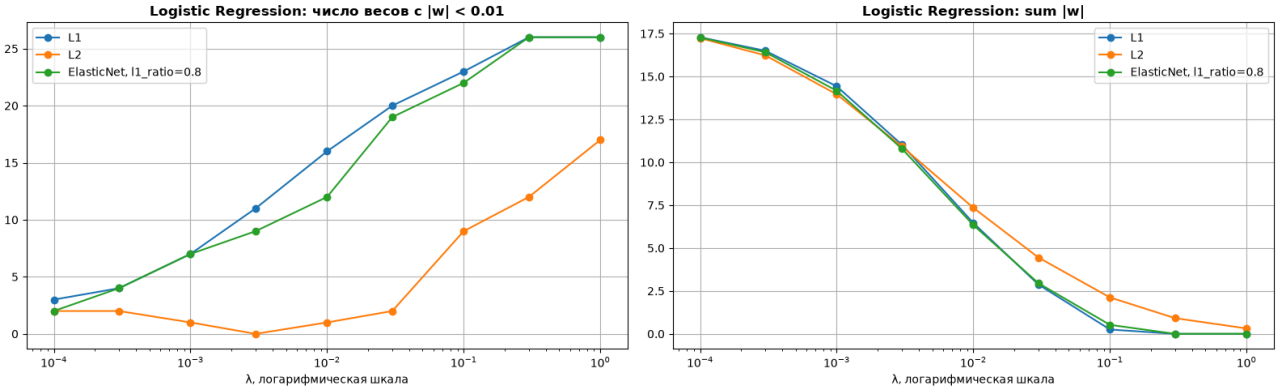


Рис. 5. Logistic Regression: число почти нулевых весов и $\sum|w|$ от λ

λ	**L1,** $w = 0$	**L2,** $w = 0$	**ElasticNet,** $w = 0$	**L1, LogLoss val**	**L2, LogLoss val**	**ElasticNet, LogLoss val**
`0.001`	`7`	`0`	`7`	`0.4502`	`0.4517`	`0.4504`
`0.01`	`16`	`0`	`12`	`0.4735`	`0.4767`	`0.4748`
`0.1`	`22`	`0`	`22`	`0.6669`	`0.5797`	`0.6424`
`1`	`26`	`0`	`26`	`0.6937`	`0.6695`	`0.6937`

В таблице логистическая регрессия, ElasticNet с `l1_ratio=0.8`. При λ=`1` L1 обнуляет все `26` весов, и LogLoss `0.6937` почти равен LogLoss константного прогноза, около `0.693`. L2 при том же λ не даёт ни одного нуля.

4.6. Вывод по части A

Регуляризация немного улучшила обе модели за счёт меньшего переобучения. У линейной регрессии `29` признаков на `439` объектов train, без регуляризации RMSE на train `737.5`, на test `870.1`. С регуляризацией RMSE на test снизился до `853.8`, у логистической регрессии LogLoss на test снизился с `0.4550` до `0.4511`, Accuracy выросла с `0.7808` до `0.7860`. По validation выбраны λ=`0.01` для линейной и λ=`0.001` для логистической регрессии во всех вариантах.

L1 и L2 по-разному действуют на веса. Градиент L1 по ненулевому весу равен $\pm\lambda$ при любом размере веса и дотягивает малые веса до нуля: при лучшем λ L1 обнулил `12` из `29` весов линейной и `7` из `26` весов логистической регрессии, а качество на test осталось таким же, как у L2. Градиент L2 $2\lambda w$ уменьшается вместе с весом, поэтому L2 уменьшает $\sum|w|$, с `8.09` до `6.27` у линейной регрессии, но ни при одном λ из сетки не обнулил ни одного веса. ElasticNet у линейной регрессии выбрал `l1_ratio=0.2` и ведёт себя как L2, у логистической выбрал `0.8` и обнулил те же `7` весов, что L1.

Среди обнулённых признаков `holiday`: в этих данных `holiday = weekday_1 + ... + weekday_5 - workingday`, и столбец не добавляет информации к остальным.

Слишком большой λ приводит к недообучению: при λ=`1` RMSE линейной регрессии на validation растёт с `738` до `1770` для L1 и до `1298` для L2.

Я бы выбрал L1: λ=`0.01` для линейной и λ=`0.001` для логистической регрессии. Качество на test то же, что у L2, а `12` и `7` признаков можно не использовать.

5. Часть B. KNN Classification

5.1. Данные

Датасет содержит `13611` зёрен фасоли и `16` числовых признаков формы, посчитанных по фотографиям: площадь, периметр, длины осей, коэффициенты формы. Пропусков нет, `68` строк-дубликатов я удалил до split, чтобы одно зерно не попало и в train, и в test. После удаления осталось `13543` строки, train - `8127`, validation - `2708`, test - `2708`.

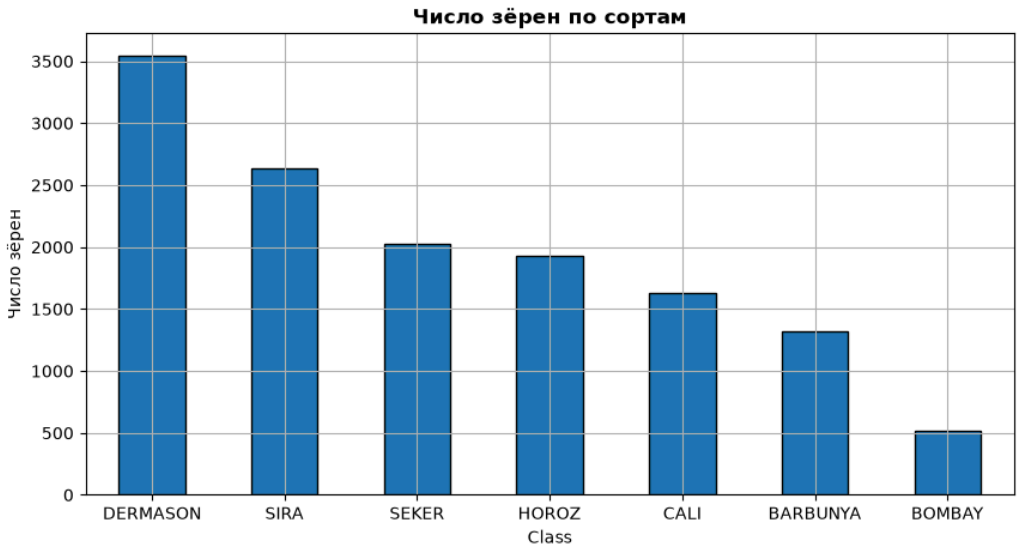


Рис. 6. Число зёрен по сортам

Классы несбалансированы: от `26.1%` у `DERMASON` до `3.8%` у `BOMBAY`, поэтому кроме Accuracy я смотрю Macro F1 - среднее F1 по классам без весов, в котором мелкие классы весят столько же, сколько крупные. Масштабы признаков различаются на восемь порядков: `Area` до `254616`, `ShapeFactor2` около `0.0017`.

Сорт	Доля	Средняя площадь	Минимальная площадь
BARBUNYA	9.7%	69804	41487
BOMBAY	3.8%	173485	114004
CALI	12.0%	75538	45504
DERMASON	26.1%	32119	20420
HOROZ	14.2%	53649	33006
SEKER	14.9%	39881	28395
SIRA	19.4%	44729	31519

5.2. Preprocessing

Категориальных признаков и пропусков нет. Целевая `Class` закодирована числами `0-6` в алфавитном порядке сортов. Все признаки стандартизированы по train. Нестандартизированные копии сохранены для сравнения.

5.3. Класс KNN

Метод	Что делает
`__init__`	сохраняет `k`, `task` (`classification` или `regression`) и `distance` (`euclidean` или `manhattan`)
`fit`	запоминает обучающую выборку, вычислений нет
`_distances`	расстояния от батча объектов до всех объектов train
`kneighbors`	индексы `k` ближайших соседей через `torch.topk`
`predict`	для классификации голосование большинством, для регрессии среднее целевой переменной соседей

Расстояния между объектами x и z:

$$d_{\text{euclidean}}(x,z) = \sqrt{\sum_j (x_j - z_j)^2}, \quad d_{\text{manhattan}}(x,z) = \sum_j |x_j - z_j|$$

```python
class KNN:
    def __init__(self, k=5, task='classification', distance='euclidean', batch_size=256):
        if task not in ('classification', 'regression'):
            raise ValueError(f'Неизвестный task: {task}')
        if distance not in ('euclidean', 'manhattan'):
            raise ValueError(f'Неизвестное расстояние: {distance}')
        self.k = k
        self.task = task
        self.distance = distance
        self.batch_size = batch_size

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y
```

```
        if self.task == 'classification':
            self.n_classes = int(y.max().item()) + 1
        return self

    def _distances(self, X):
        diff = X[:, None, :] - self.X_train[None, :, :] # (batch, n_train, n_features)
        if self.distance == 'euclidean':
            return torch.sqrt((diff ** 2).sum(dim=2))
        return diff.abs().sum(dim=2)

    def kneighbors(self, X):
        idx = []
        for start in range(0, X.shape[0], self.batch_size):
            d = self._distances(X[start:start + self.batch_size])
            idx.append(torch.topk(d, self.k, dim=1, largest=False).indices) # k наименьших
расстояний
        return torch.cat(idx)

    def predict(self, X):
        neighbors_y = self.y_train[self.kneighbors(X)] # (n, k) целевая переменная соседей
        if self.task == 'classification':
            votes = torch.zeros(X.shape[0], self.n_classes)
            votes.scatter_add_(1, neighbors_y, torch.ones(neighbors_y.shape)) # число голосов за
каждый класс
            return votes.argmax(dim=1)
        return neighbors_y.float().mean(dim=1)
```

Расстояния считаются батчами по `256` объектов, чтобы матрица разностей размером «батч × `n_train` × число признаков» помещалась в память. Голоса считаются через `scatter_add_`: каждый сосед добавляет `1` своему классу, побеждает `argmax`. При равенстве голосов побеждает класс с меньшим номером.

`fit` только сохраняет данные и выполняется мгновенно. `predict` считает расстояние от каждого объекта до каждого объекта train, сложность $O(n_{\text{test}} \cdot n_{\text{train}} \cdot d)$, где d - число признаков.

5.4. Подбор k

Значения `k`: `1`, `3`, `5`, `7`, `9`, `11`, `15`, `21`, `31`, `51`, `75`, `101`. Для каждой комбинации расстояния и scaling `k` выбирается по Macro F1 на validation.

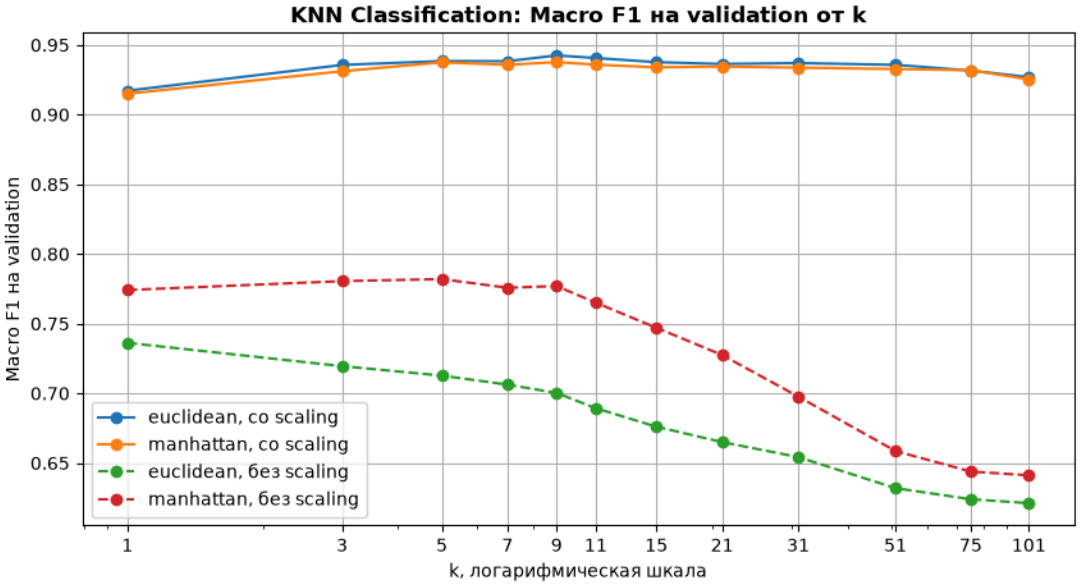


Рис. 7. Macro F1 на validation от `k`

Конфигурация	**k**	**Macro F1 val**	**Accuracy test**	**Macro F1 test**
Euclidean, со scaling	9	`0.9425`	`0.9173`	`0.9297`
Manhattan, со scaling	9	`0.9377`	`0.9162`	`0.9288`
Euclidean, без scaling	1	`0.7362`	`0.7190`	`0.7225`
Manhattan, без scaling	5	`0.7819`	`0.7725`	`0.7759`

Итоговая модель: Euclidean, `k=9`, со scaling. На test Accuracy `0.9173`, Macro F1 `0.9297`.

5.5. Confusion matrix

Строка - истинный сорт, столбец - предсказанный, на диагонали верные ответы.

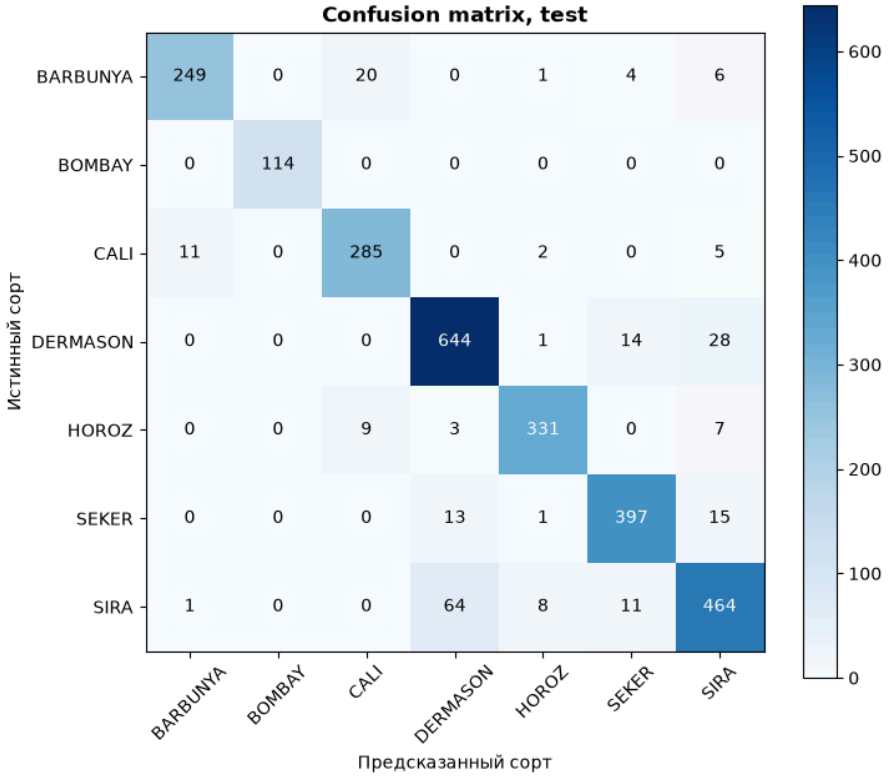


Рис. 8. Confusion matrix итоговой модели на test

Сорт	Recall	Precision	F1
BARBUNYA	0.8893	0.9540	0.9205
BOMBAY	1.0000	1.0000	1.0000
CALI	0.9406	0.9076	0.9238
DERMASON	0.9374	0.8895	0.9128
HOROZ	0.9457	0.9622	0.9539
SEKER	0.9319	0.9319	0.9319
SIRA	0.8467	0.8838	0.8649

5.6. Вывод по части B

Главный фактор качества KNN на этих данных - scaling. Он поднимает Macro F1 на test с `0.7225` до `0.9297` у Euclidean и с `0.7759` до `0.9288` у Manhattan. Без стандартизации соседи выбираются почти только по `Area` и `ConvexArea` со стандартным отклонением около `29000`, признаки формы в расстояние почти не попадают. Без scaling качество падает с ростом `k`, у Euclidean с `0.7362` при `k=1` до `0.6212` при `k=101`: среди зёрен близкого размера много зёрен других сортов.

Со scaling Euclidean и Manhattan почти не различаются, Macro F1 на test `0.9297` и `0.9288`. Без scaling Manhattan лучше на `0.046` на validation: Euclidean возводит разности в квадрат, поэтому доля крупного признака в расстоянии у него больше.

При k=1 Macro F1 на validation 0.9172, модель повторяет шум отдельных зёрен. Максимум 0.9425 достигается при k=9, при k=101 качество падает до 0.9269, границы классов размываются.

BOMBAY распознан без ошибок, 114 из 114, это самые крупные зёрна. Больше всего ошибок между SIRA и DERMASON: 64 и 28 зёрен, у SIRA самый низкий F1 0.8649. По площади эту пару не объяснить, причину я не проверял. Пара BARBUNYA и CALI даёт 20 и 11 ошибок, их средние площади 69804 и 75538 ближе, чем у любой другой пары сортов.

6. Часть C. KNN Regression

6.1. Данные

Датасет содержит 8760 часов с декабря 2017 по ноябрь 2018 года: число аренд велосипедов в Сеуле и погоду в этот час. Пропусков и дубликатов нет. Распределение Rented Bike Count скошено вправо: медиана 504.5, среднее 704.6, максимум 3556. По часам видны два пика спроса, в 8 и 18 часов.

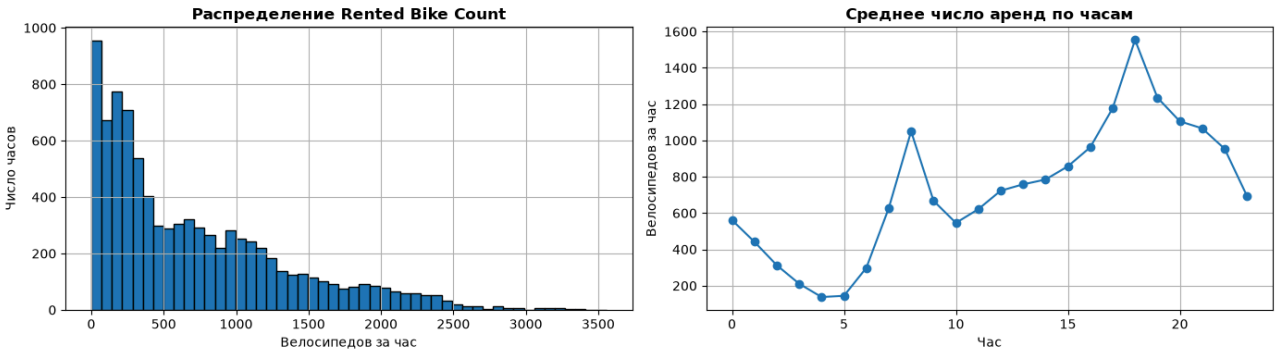


Рис. 9. Распределение числа аренд и среднее число аренд по часам

6.2. Preprocessing

Признаки	**Решение**
Functioning Day	удалены 295 часов, когда прокат не работал: в них число аренд всегда 0, спрос не измерялся. Сам признак после этого не нужен
Date	из даты взят признак выходного дня Weekend, дата удалена
Hour	заменён на HourSin и HourCos, чтобы 23 и 0 часов оказались рядом, а не на расстоянии 23
Seasons	one-hot кодирование, первая категория отбрасывается
Holiday	переведён в 0 и 1
все 15 признаков	стандартизация по train, включая бинарные, чтобы каждый признак вносил в расстояние вклад одного масштаба

После удаления осталось 8465 строк, train - 5079, validation - 1693, test - 1693.

6.3. Подбор k

Используется тот же класс KNN с task='regression': прогноз - среднее число аренд у k ближайших соседей. k выбирается по RMSE на validation из того же списка, что в части B.

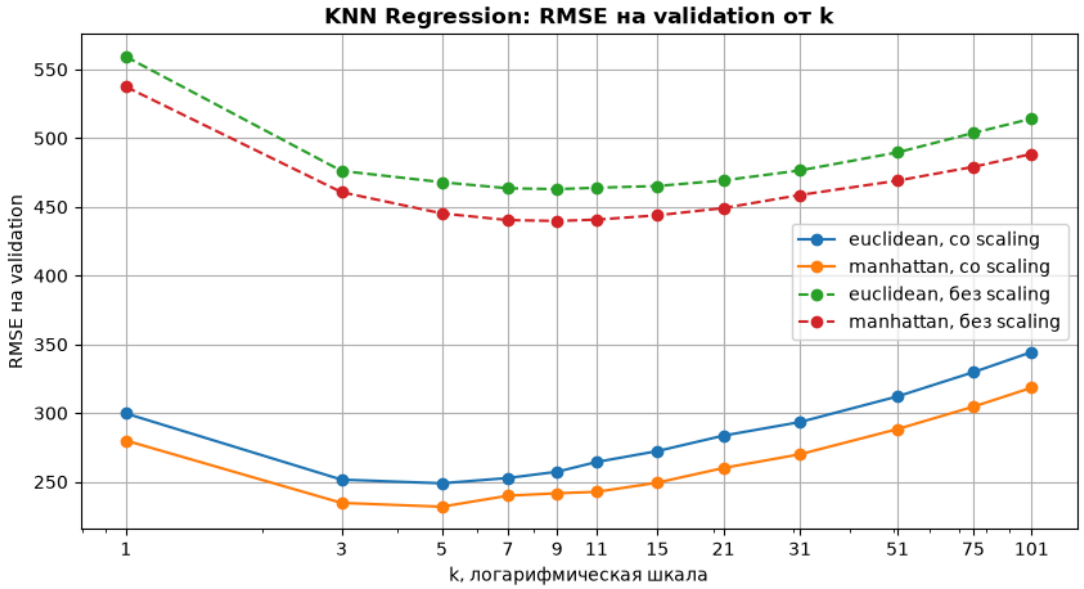


Рис. 10. RMSE на validation от k

6.4. Результаты

Основной baseline, как в лабораторной №1, предсказывает всем объектам среднее по train. Второй baseline предсказывает среднее по train для того же часа суток.

Модель	k	MAE test	RMSE test	R^2 test
KNN, Manhattan, со scaling	5	154.4	250.4	0.8574
KNN, Euclidean, со scaling	5	161.1	262.3	0.8434
KNN, Manhattan, без scaling	9	312.5	444.0	0.5515
KNN, Euclidean, без scaling	9	334.2	469.4	0.4987
Baseline, среднее train	-	531.5	663.1	-0.0002
Baseline, среднее по часу	-	418.3	540.6	0.3352

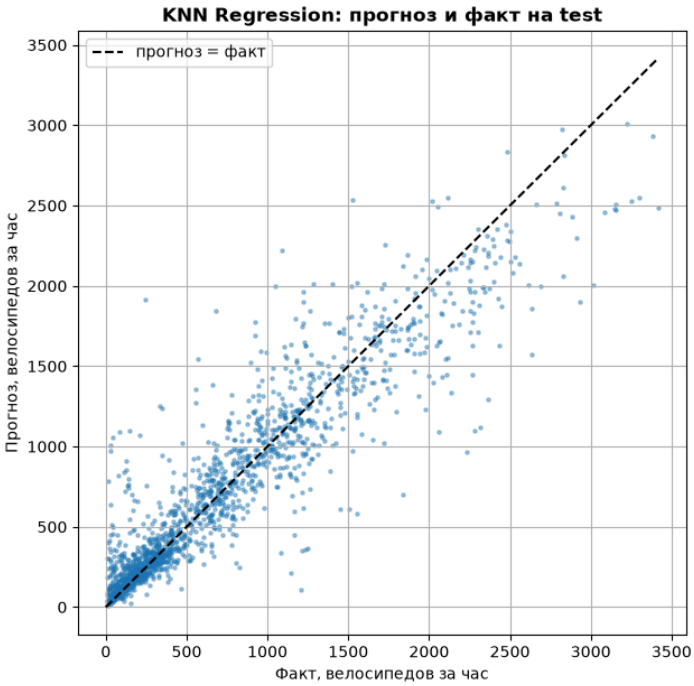


Рис. 11. Прогноз итоговой модели и факт на test

R^2 константного baseline чуть меньше нуля, потому что среднее в train `731.7` выше, чем в test `722.1`.

6.5. Вывод по части C

Итоговая модель - Manhattan, `k=5`, со scaling. Она снижает MAE на test в `3.4` раза и RMSE в `2.6` раза относительно среднего, а RMSE относительно среднего по часу - в `2.2` раза. Час объясняет треть дисперсии спроса, остальное KNN берёт из погоды, сезона и типа дня.

Без scaling R^2 падает с `0.8574` до `0.5515`. В расстоянии доминируют `Visibility` со стандартным отклонением `609` и `Humidity` с `20`, а час закодирован числами от `-1` до `1` и почти не влияет на выбор соседей.

Manhattan лучше Euclidean в обоих режимах, со scaling RMSE на test `250.4` против `262.3`. Причину я не проверял. Возможно, Euclidean сильнее штрафует большое расхождение по одному признаку, например по `Rainfall` или `Snowfall` с редкими большими значениями.

Лучший `k` маленький: RMSE на validation `280.0` при `k=1`, `231.9` при `k=5` и `318.5` при `k=101` для Manhattan. Спрос резко меняется в течение дня, и большое число соседей смешивает разные часы. Кроме того, при случайном split в train лежат соседние часы того же дня с почти той же погодой. Вероятно, это завышает качество KNN, split по дням я не проверял.

7. Вывод

Я добавил L1, L2 и ElasticNet в модели из лабораторной №1, реализовал KNN для классификации и регрессии и для каждой части подобрал параметры по validation.

Часть	**Итог на test**
A. Regularization	небольшое улучшение: RMSE `870.1` → `853.8`, LogLoss `0.4550` → `0.4511`. L1 обнулил `12` весов из `29` и `7` из `26` без потери качества, L2 не обнулил ни одного
B. KNN Classification	Euclidean, `k=9`, со scaling: Accuracy `0.9173`, Macro F1 `0.9297`
C. KNN Regression	Manhattan, `k=5`, со scaling: RMSE `250.4` против `663.1` у среднего и `540.6` у среднего по часу

Регуляризация уменьшила переобучение линейной регрессии на маленькой выборке и дала около `2%` по RMSE, а L1 вдобавок убрал лишние признаки. Для KNN важнее всего scaling: без него Macro F1 в части B падает до `0.7759`, R^2 в части C до `0.5515`.

Что я бы улучшил:

- В части C разбить данные по дням, чтобы соседние часы одного дня не попадали в train и test одновременно.
- В KNN разрешать равенство голосов в пользу ближайшего соседа и взвешивать голоса по расстоянию.
- Ускорить подбор `k`: один раз найти `max(k)` соседей и брать первые `k`, а не пересчитывать расстояния для каждого `k`.
- Для больших выборок заменить полный перебор на KD-tree или приближённый поиск соседей.